

Practical - 1

Aim: - Study of Big Data

Definition of Big Data

Big Data describes very large, complex, and continuously expanding datasets that cannot be effectively collected, stored, managed, or examined using conventional data management methods. It is not limited to data size alone; Big Data also involves how fast data is produced, the different forms it takes, and how effectively it can be transformed into valuable insights.

Characteristics of Big Data

Big Data is commonly explained using its main features, known as the V's of Big Data. These characteristics highlight why traditional systems struggle to handle Big Data efficiently.

1. Volume

Volume indicates the enormous amount of data created from multiple sources such as social networks, sensors, online activities, and smart devices. This information is generated nonstop and is typically measured in units like terabytes, petabytes, or even exabytes. Managing such massive datasets requires distributed storage and scalable computing platforms.

2. Velocity

Velocity refers to the rate at which data is produced, transmitted, and processed. In many real-world scenarios, data must be analyzed instantly or with minimal delay to remain useful. Common examples include digital payment authorization, live market feeds, navigation systems, and real-time surveillance or monitoring.

3. Variety

Variety represents the wide range of data formats present in Big Data environments. This includes structured data from databases, semi-structured formats such as XML and JSON, and unstructured content like emails, images, videos, and sound files. Handling and combining these different data forms presents a major challenge in Big Data analytics.

4. Veracity

Veracity focuses on reliability, accuracy, and quality of data. Because Big Data often comes from many sources and is gathered in real time, it may contain errors, duplicates, or missing values. Applying proper data cleansing and validation techniques is crucial to ensure dependable analysis and informed decision-making.

5. Value

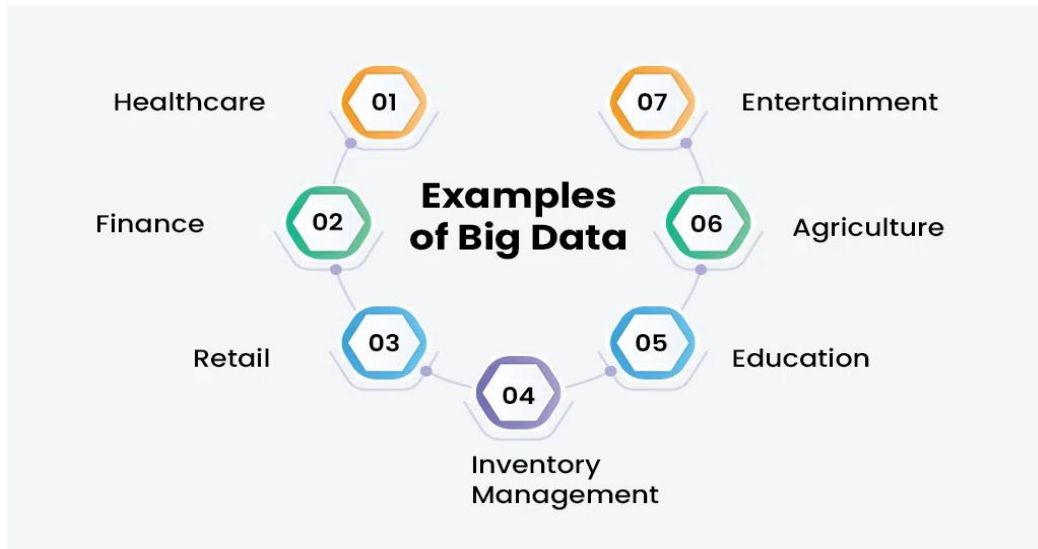
Value highlights the importance of converting raw data into meaningful outcomes. Simply storing large datasets has little benefit unless insights can be extracted to guide decisions, enhance productivity, or forecast future patterns. Value-oriented analytics enables organizations to gain advantages and optimize performance.

6. Variability

Variability refers to fluctuations in data generation and changes in data interpretation over time. The volume and speed of data can rise or fall based on situations, seasons, or user actions. For instance, online activity increases during major events, and purchasing behavior shifts during holidays or sales periods, adding complexity to analysis.

Real-Time Examples of Big Data

Real-time Big Data refers to information that is produced continuously and analyzed instantly to deliver immediate insights or trigger quick actions. In the modern digital environment, many sectors depend on real-time Big Data to enhance operational efficiency, support better decisions, and improve customer experience.



Real-Time Examples(Fig 1)

1. Social Media Platforms

Social networking platforms create enormous volumes of data every second through user activities such as posts, reactions, comments, shares, photos, and videos.

2. Online Banking and Digital Payments

Banks and financial services handle huge numbers of daily transactions using credit cards, debit cards, UPI, and internet banking systems. This real-time data helps detect fraud, verify payments, and ensure secure transfers.

3. E-Commerce Websites

Online shopping platforms capture live data related to browsing behavior, clicks, shopping carts, and completed purchases. This information is used to provide instant recommendations, adjust prices dynamically, and offer personalized deals.

4. Healthcare Monitoring Systems

In the medical field, smart devices and hospital equipment continuously generate real-time information such as pulse rate, blood pressure, oxygen saturation, and other patient health indicators.

5. Smart Cities and Traffic Management

Smart city infrastructures rely on sensors, cameras, and GPS technologies to gather live data on traffic movement, air quality, weather updates, and public transportation activity.

6. Entertainment and Streaming Platforms

Streaming services collect real-time data based on what users watch, how long they watch, their likes, and content preferences. This data is used to instantly suggest movies, series, and music, increasing user engagement and satisfaction.

7. Transportation and Ride-Sharing Services

Ride-hailing and navigation applications use live GPS data to monitor vehicle positions, predict arrival times, determine fares, and recommend the most efficient routes.

Real-Time Generation of Big Data

Big Data is produced nonstop from many digital sources as humans, machines, and systems interact with technology every moment. The fast expansion of the internet, smartphones, cloud platforms, and intelligent systems has greatly increased how much data is created, how quickly it appears, and the different forms it takes.

1. Social Media Platforms

Social platforms generate large volumes of data from activities such as posting updates, sharing reels, reacting to content, participating in polls, joining groups, and sending direct messages—far beyond simple likes and comments.

2. Internet of Things (IoT) Devices

Connected devices like smart meters, wearable health trackers, home security systems, factory sensors, and autonomous vehicles constantly send status updates and environmental readings, creating continuous data streams.

3. Online Transactions

Digital interactions such as mobile recharges, subscription payments, hotel reservations, food delivery orders, and online registrations also contribute to Big Data, along with traditional banking and shopping transactions.

4. Web and Application Analytics

Modern websites and apps collect analytical data including session duration, navigation paths, button interactions, crash reports, and performance metrics, helping organizations understand user behavior and system health.

5. Multimedia and User-Generated Content

Apart from images and videos, podcasts, voice notes, screen recordings, and live gaming streams significantly add to Big Data, especially as content creation becomes more common on mobile devices.

6. Sensors, Robotics, and Automated Systems

Robots, production lines, drones, satellites, and laboratory instruments continuously produce operational data. This information is widely used for automation control, quality inspection, disaster prediction, and advanced research.

7. Mobile Devices and Location-Based Services

Smartphones generate data not only from calls and apps but also from biometric sensors, motion tracking, QR scans, and location-based services such as ride booking and nearby recommendations.

Use of Big Data

Big Data is applied to study massive and complex datasets to uncover meaningful patterns, emerging trends, and hidden connections. These insights support smarter decision-making, streamline operations, and help organizations stay competitive.

1. Predictive Analysis

Big Data helps organizations anticipate future outcomes by examining past records along with live data streams. It is commonly used for sales forecasting, trend analysis, capacity planning, and early detection of equipment failures.

2. Customer Behavior Analysis

By studying customer information such as buying habits, online activity, and reviews, companies gain a deeper understanding of user needs. This supports better marketing campaigns, improved products, and stronger customer relationships.

3. Fraud Detection

Big Data techniques are used to spot unusual behavior and potential threats in real time. This is particularly important in banking, finance, and digital payment platforms to reduce fraud and enhance transaction security.

4. Performance Optimization

Organizations rely on Big Data to track application and system performance, locate inefficiencies, and improve workflows. This leads to better resource utilization, higher productivity, and lower operational expenses.

5. Personalization of Services

Using Big Data, businesses can deliver customized experiences such as tailored recommendations, personalized content, and targeted promotions based on individual user interests and activity patterns.

6. Risk Management

Big Data supports the identification and evaluation of possible risks, allowing organizations to take proactive measures. It is widely applied in finance, insurance, and strategic planning to reduce uncertainty and limit potential losses.

How Industries are Utilizing Big Data

Organizations across different sectors use Big Data to process vast amounts of information coming from users, devices, and digital platforms. With advanced analytics, industries can improve planning, automate operations, reduce waste, and create smarter services.

1. Healthcare Industry

Healthcare providers use Big Data to manage electronic health records, analyze lab reports, and track real-time data from medical devices. It supports early diagnosis, remote patient care, personalized medicine, and clinical decision support. Big Data is also applied to optimize hospital staffing and resource allocation.

2. Banking and Financial Services

Financial institutions apply Big Data for transaction analysis, customer segmentation, regulatory compliance, and fraud prevention. Real-time monitoring helps flag suspicious behavior quickly, while historical data is used for loan approval, portfolio management, and financial forecasting.

3. Retail and E-Commerce

Retail companies use Big Data to study buying trends, improve warehouse operations, predict seasonal demand, and design loyalty programs. It also helps optimize product placement, manage returns, and enhance after-sales support.

4. Manufacturing Industry

Manufacturers rely on Big Data for production planning, energy optimization, and supply chain visibility. Machine data helps identify inefficiencies, improve product consistency, and automate quality inspections, leading to faster production cycles.

5. Transportation and Logistics

Logistics providers use Big Data for shipment tracking, warehouse coordination, driver performance analysis, and delivery scheduling. Data-driven insights help minimize delays, improve route planning, and reduce fuel consumption.

6. Education Sector

Educational organizations apply Big Data to evaluate course effectiveness, predict student dropouts, and design adaptive learning systems. It also assists administrators in curriculum planning and resource management.

7. Government and Smart Cities

Governments utilize Big Data for population analysis, policy planning, disaster preparedness, and digital governance. Data from public services supports smarter energy usage, improved urban planning, and faster response to emergencies.

Future of Big Data

1. Accelerated Growth of Data Creation

In the coming years, data generation will rise dramatically due to the increasing use of smartphones, online platforms, connected devices, cloud platforms, and digital services. Data will be produced not only by people but also by automated systems and smart machines, resulting in massive datasets that demand more powerful Big Data storage and processing solutions.

2. Deeper Integration with AI and Machine Learning

Big Data will work even more closely with Artificial Intelligence and Machine Learning. These technologies will process huge datasets to discover patterns automatically, improve predictions, and support intelligent decision-making. This will enable advanced applications such as smart assistants, self-driving systems, intelligent recommendation engines, and data-driven healthcare.

3. Shift Toward Real-Time and Forecast-Based Analytics

The future of Big Data will emphasize instant analysis and future-oriented insights. Organizations will increasingly act on data as it arrives—supporting real-time monitoring, fraud prevention, and customer support. At the same time, predictive models will help anticipate market trends, operational risks, and user needs with greater accuracy.

4. Edge Computing and High-Speed Data Processing

As edge computing expands, more data will be processed near its source instead of only in centralized clouds. This approach will reduce delays, increase responsiveness, and improve efficiency, especially in areas like industrial automation, connected vehicles, and smart infrastructure.

5. Stronger Focus on Security, Privacy, and Data Management

With the growing importance of Big Data, protecting information will become a top priority. Future systems will adopt advanced security measures such as enhanced encryption, strict access controls, and well-defined governance policies to safeguard sensitive data and meet privacy regulations.

6. Wider Adoption Across Sectors

Big Data will continue spreading into nearly every field, including healthcare, banking, education, manufacturing, transportation, agriculture, and public services. It will drive digital transformation, support automation, and encourage innovation, helping organizations operate more intelligently and competitively.

7. Rising Career Demand and New Skill Sets

The demand for professionals like data analysts, data engineers, data scientists, and AI specialists will keep increasing. Big Data will remain a high-growth domain, creating new roles and requiring a mix of technical, analytical, and problem-solving skills.

Practical-2

Aim:- To Implement BeautifulSoup to extract data from any given website

Code:

```
import requests
from bs4 import BeautifulSoup
import csv

URL = "http://quotes.toscrape.com"

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
}

try:
    response = requests.get(URL, headers=headers)
    response.raise_for_status()
    html_content = response.text
except requests.exceptions.RequestException as e:
    print(f"Error fetching the URL: {e}")
    exit()

soup = BeautifulSoup(html_content, "html.parser")

page_title = soup.title.string
print("Page Title:", page_title)

quotes = soup.find_all("div", class_="quote")

data = []

for q in quotes:
    text = q.find("span", class_="text").text
    author = q.find("small", class_="author").text
    tags = [tag.text for tag in q.find_all("a", class_="tag")]

    print(f"Quote: {text}")
    print(f"Author: {author}")
    print(f"Tags: {' '.join(tags)}\n")

    data.append([text, author, " ".join(tags)])

with open("quotes.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow(["Quote", "Author", "Tags"])
    writer.writerows(data)

print("Data saved to quotes.csv")
```

OUTPUT:

```
PS C:\Users\user\Desktop\BD 4> & "C:/Users/user/Desktop/BD 4/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\user\Desktop\BD 4> & "C:/Users/user/Desktop/BD 4/.venv/Scripts/python.exe"
D 4/prac2.py"
CSV file created successfully: quotes.csv
(.venv) PS C:\Users\user\Desktop\BD 4>
```

```
quotes.csv > data
1 Quote,Author,Tags
2 "The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking.",Albert Einstein,"change, deep-thoughts, thinking, world"
3 "It is our choices, Harry, that show what we truly are, far more than our abilities.",J.K. Rowling,"abilities, choices"
4 "There are only two ways to live your life. One is as though nothing is a miracle. The other is as though everything is a miracle.",Albert Einstein,"inspirational, life, live, miracle, miracles"
5 "The person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably stupid.",Jane Austen,"aliteracy, books, classic, humor"
6 "Imperfection is beauty, madness is genius and it's better to be absolutely ridiculous than absolutely boring.",Marilyn Monroe,"be-yourself, inspirational"
7 "Try not to become a man of success. Rather become a man of value.",Albert Einstein,"adulthood, success, value"
8 "It is better to be hated for what you are than to be loved for what you are not.",André Gide,"life, love"
9 "I have not failed. I've just found 10,000 ways that won't work.",Thomas A. Edison,"edison, failure, inspirational, paraphrased"
10 "A woman is like a tea bag; you never know how strong it is until it's in hot water.",Eleanor Roosevelt,misattributed-eleanor-roosevelt
11 "A day without sunshine is like, you know, night.",Steve Martin,"humor, obvious, simile"
12 |
```


Practical-3

Aim: - To study Hadoop and understand its architecture, components, features, and applications.

Why Hadoop?

Traditional data systems faced major difficulties when:

- The volume of data increased rapidly (social media, IoT devices, system logs)
- Data types became mostly unstructured (images, videos, text files)
- Vertical scaling (using more powerful servers) became very costly

Hadoop was developed to address these challenges:

Scalability

- Supports horizontal scaling by adding more machines
- Reduces dependency on expensive high-end hardware

Fault Tolerance

- Data is stored with replication (default: 3 copies)
- System automatically recovers if a node crashes

Cost Efficiency

- Works on low-cost commodity hardware
- Available as open-source software

Flexibility

- Can process both structured and unstructured data
- Uses schema-on-read (structure applied when data is accessed)

What is Hadoop?

Apache Hadoop is an open-source framework used to store and process very large datasets across clusters of commodity hardware (standard servers).

It is designed to manage:

- Extremely large datasets (from terabytes to petabytes)
- Structured, semi-structured, and unstructured data formats
- Distributed storage with parallel data processing
- Fault tolerance with automatic recovery from node failures

Main Components of Hadoop

1. **HDFS (Hadoop Distributed File System)** – Responsible for distributed data storage
2. **YARN (Yet Another Resource Negotiator)** – Handles cluster resource management
3. **MapReduce** – Framework used for distributed data processing
4. **Hadoop Common** – Collection of libraries and utilities required by Hadoop components

Features of Hadoop



1. Scalability

Hadoop can easily scale by adding more machines to the cluster. This allows it to handle very large datasets efficiently.

2. Reliability

Hadoop stores multiple copies of data across different nodes. If one node fails, the data can still be accessed from another node.

3. Flexibility

Hadoop can process different types of data such as structured, semi-structured, and unstructured data. This makes it suitable for many big data applications.

4. Commodity Hardware

Hadoop runs on inexpensive, regular hardware instead of costly high-end servers. This reduces the overall infrastructure cost.

5. Open Source

Hadoop is free and open-source software maintained by the Apache Software Foundation. Anyone can use, modify, and improve it.

6. Big Data Analytics

Hadoop can process and analyze extremely large datasets efficiently. It uses distributed processing to speed up data analysis.

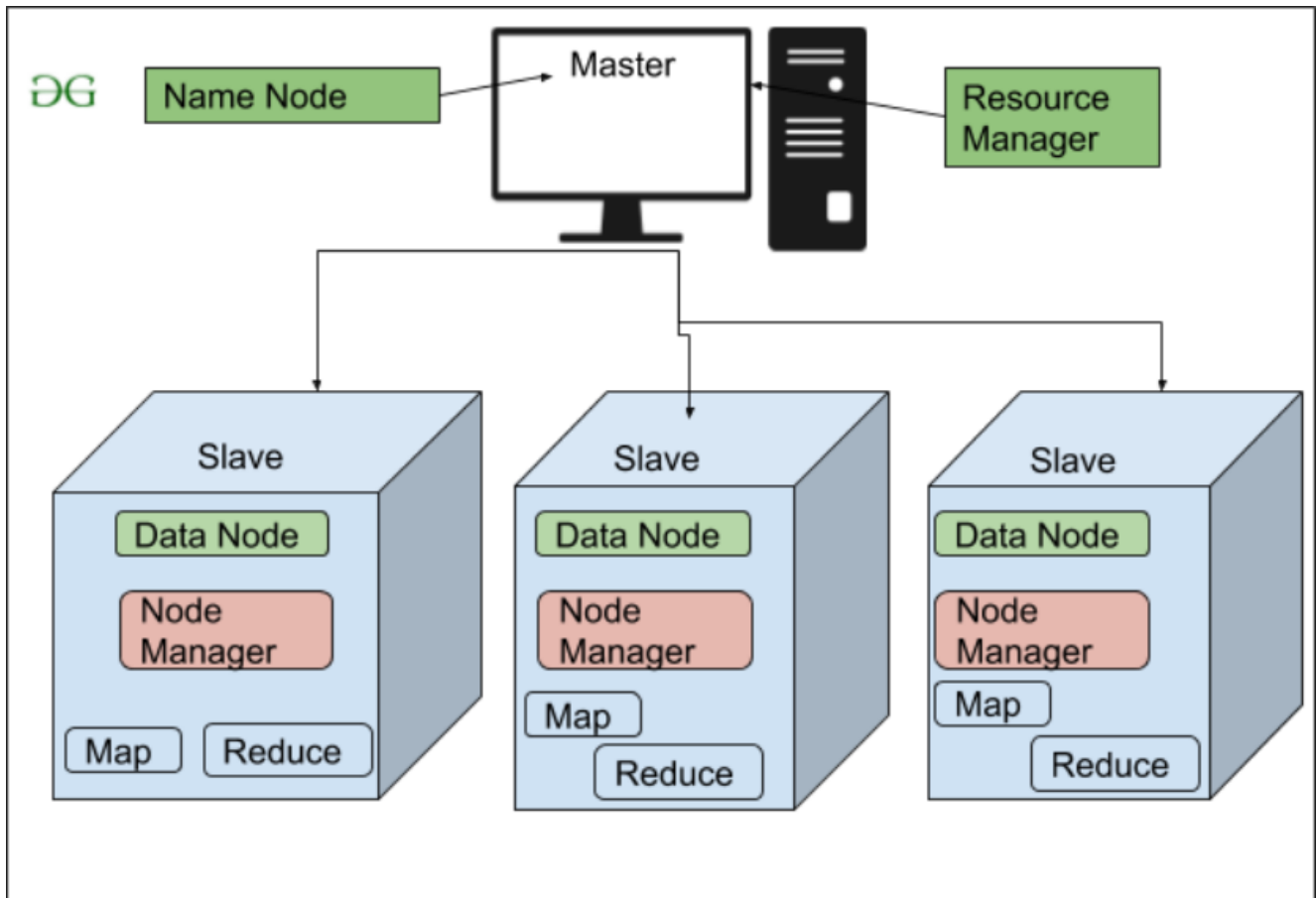
7. Ecosystem Support

Hadoop works with many tools like Hive, Pig, and HBase. These tools help in data storage, querying, and analysis.

8. Distributed File System

Hadoop uses HDFS to store data across multiple machines. This allows large files to be stored and accessed efficiently in a distributed environment.

Hadoop Architecture Overview



Hadoop uses a **master-slave architecture** where the master node manages the cluster and slave nodes perform storage and processing tasks. All machines in the cluster work together as a single distributed system.

The architecture consists of:

- **Master Node**
- **Slave Nodes**

1. Master Nodes

Master nodes are responsible for managing the cluster, coordinating tasks, and maintaining metadata.

a) NameNode

The **NameNode** is the central component of HDFS (Hadoop Distributed File System). It manages and maintains metadata about the file system.

The NameNode:

- Keeps track of file names and directory structure
- Maintains information about data blocks
- Stores the mapping of blocks to DataNodes
- Controls access permissions

Important:

The NameNode does **not store actual data**. It only stores metadata information about where the data is located.

If the NameNode fails, the entire cluster becomes inaccessible (unless a Secondary/Standby NameNode is configured).

b) ResourceManager

The **ResourceManager** is part of YARN (Yet Another Resource Negotiator). It manages cluster resources such as CPU and memory.

The ResourceManager:

- Allocates resources to applications
- Schedules jobs across nodes
- Monitors running applications
- Ensures fair distribution of workload

It acts as the central authority for resource management in the cluster.

2. Slave Nodes

Slave nodes perform the actual work of storing and processing data.

a) DataNode

The **DataNode** is responsible for storing the actual data blocks in HDFS.

Each DataNode:

- Stores data blocks
- Performs read/write operations
- Sends heartbeat signals to the NameNode
- Replicates data blocks as instructed

DataNodes regularly communicate with the NameNode to confirm they are functioning properly.

b) NodeManager

The **NodeManager** is responsible for executing tasks on each slave node.

The NodeManager:

- Manages containers (execution units)
- Monitors resource usage
- Reports status to the ResourceManager
- Executes Map and Reduce tasks

It ensures that tasks are completed properly on that particular node.

Advantages of Hadoop

1. Scalability

Hadoop can easily scale by adding more machines to the cluster. This allows it to process very large amounts of data efficiently.

2. Cost Effective

Hadoop runs on low-cost commodity hardware instead of expensive high-end servers, making it affordable for large-scale data processing.

3. Fault Tolerance

Hadoop automatically replicates data across multiple nodes. If one node fails, the data can still be accessed from another node.

4. Flexibility

Hadoop can handle structured, semi-structured, and unstructured data such as text, images, and videos.

5. Distributed Processing

Hadoop processes data in parallel across multiple nodes, which improves speed and efficiency when working with big data.

6. Open Source

Hadoop is free and open-source software supported by the Apache community, allowing organizations to use and modify it easily.

Disadvantages of Hadoop

1. Complex Setup

Hadoop installation and configuration can be complicated, especially for large clusters.

2. Security Issues

Earlier versions of Hadoop had limited security features, making data protection more challenging.

3. High Maintenance

Managing and maintaining many nodes in a cluster requires skilled administrators.

4. Not Suitable for Small Data

Hadoop is designed for very large datasets, so using it for small data can be inefficient.

5. High Latency

Hadoop is mainly designed for batch processing, so it is not ideal for real-time data processing.

6. NameNode Dependency

If the NameNode fails, the entire Hadoop cluster may become inaccessible unless a backup system is used.

Applications of Hadoop

1. Big Data Analytics

Hadoop is used to process and analyze very large datasets to discover patterns, trends, and useful insights.

2. Social Media Analysis

Companies analyze data from social media platforms to understand user behavior, trends, and customer preferences.

3. Log Processing

Hadoop is used to store and analyze server logs, application logs, and system logs for monitoring and troubleshooting.

4. Recommendation Systems

Organizations use Hadoop to build recommendation systems for products, movies, and content based on user behavior.

5. Fraud Detection

Banks and financial institutions use Hadoop to analyze transaction data and detect fraudulent activities.

6. Data Warehousing

Hadoop is used as a data warehouse to store and manage large volumes of structured and unstructured data.

Comparison

RDBMS	Hadoop
RDBMS is a relational database management system.	Hadoop is a node based flat structure.
RDBMS is used for OLTP processing.	Hadoop is used for analytical and for big data processing.
In RDBMS, the database cluster uses the same data files stored in shared storage.	In Hadoop, the storage data can be stored independently in each processing node.
In RDBMS, preprocessing of data is required before storing it.	In Hadoop, you don't need to preprocess data before storing it.

Practical-4

Aim: - To study the MapReduce programming model in Hadoop and understand its working mechanism.

MapReduce

MapReduce is a **distributed programming model in Apache Hadoop** used to process very large datasets across a cluster of computers. It allows data to be processed in parallel by dividing the work among multiple machines, which improves efficiency and speed.

The basic idea of MapReduce is:

Divide the data → Process it in parallel → Combine the results to produce the final output.

It consists of two main phases:

1. Mapper

The Mapper is the first phase of MapReduce.

- It takes input data in the form of key-value pairs (k, v).
- It processes each input record.
- It generates intermediate key-value pairs (k', v').

The Mapper does filtering, sorting, and basic transformations.

2. Reducer

The Reducer is the second phase.

- It receives grouped key-value pairs from the mapper phase.
- It performs aggregation or computation on values.
- It produces the final output.

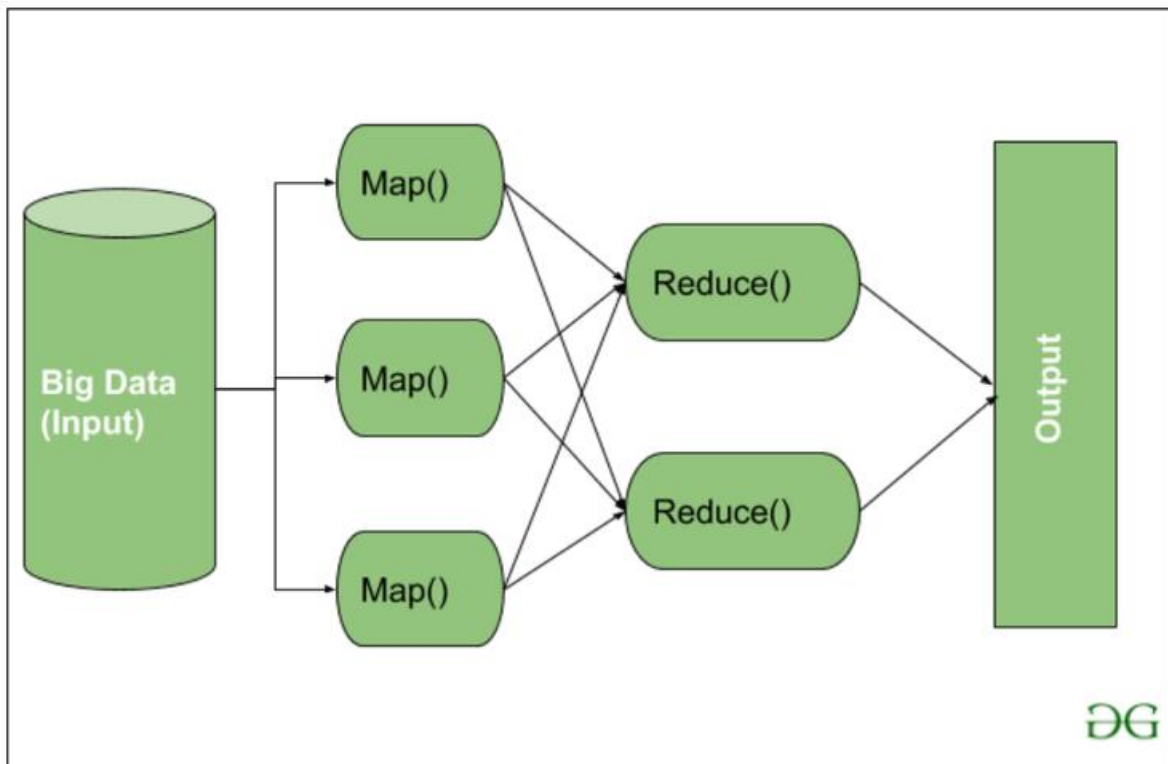
The output is stored back in HDFS.

Between these phases there is an important step:

- **Shuffle & Sort** – The system groups intermediate data by key and sorts it so that all values related to the same key are sent to the same reducer for processing.

Data Flow in MapReduce (Map → Shuffle → Reduce)

Let's understand step-by-step how data moves inside a Hadoop cluster.



Step 1: Input Phase

The input data is stored in **HDFS**. Large files are automatically divided into smaller blocks (default **128 MB**), and each block is processed by a separate **Mapper**. This allows Hadoop to process data in parallel.

Example Input:

apple banana apple
banana mango apple

Step 2: Map Phase

In this phase, the **Mapper reads the input data line by line** and converts it into **key–value pairs**. Each mapper works independently on its assigned data.

Example Output:

(apple,1)
(banana,1)
(apple,1)

Step 3: Shuffle & Sort Phase (Heart of MapReduce)

This phase is handled automatically by Hadoop. The intermediate data from mappers is **grouped by key and sorted**, and then sent to reducers.

Before Shuffle:

(apple,1)
(banana,1)
(apple,1)

After Shuffle:

apple → [1,1]

banana → [1]

This step includes **partitioning, sorting, and transferring data to reducers**.

Step 4: Reduce Phase

The **Reducer processes the grouped values** and combines them to produce the final result.

Input:

apple → [1,1]

banana → [1]

Output:

(apple,2)

(banana,1)

The final output is stored back in **HDFS**.

MapRed vs MapReduce APIs

Feature	MapRed API (Old API)	MapReduce API (New API)
Package	org.apache.hadoop.mapred	org.apache.hadoop.mapreduce
Introduction	Older Hadoop API	Improved and newer API
Ease of Use	More complex and less flexible	Easier to use and more flexible
Object Model	Uses older programming model	Uses better object-oriented design
Performance	Less efficient compared to new API	More optimized and efficient
Status	Deprecated (not recommended)	Actively used and recommended

Example: Total Sales per Product using MapReduce

Find the **total sales for each product** from a sales dataset.

Example Input (Sales Dataset)

Laptop, 500

Mobile, 300

Laptop, 700
Tablet, 200
Mobile, 400

Each line → (**Product, Sales Amount**)

Step 1: Splitting

Hadoop divides the input file into smaller blocks and distributes them across different nodes so that they can be processed in parallel.

Step 2: Mapping

The **Mapper** reads each line and outputs **key–value pairs**.

Mapper Output:

(Laptop, 500)
(Mobile, 300)
(Laptop, 700)
(Tablet, 200)
(Mobile, 400)

So:

key → **Product**

value → **Sales Amount**

Step 3: Shuffling

Hadoop automatically groups the same keys together.

(Laptop, [500, 700])
(Mobile, [300, 400])
(Tablet, [200])

Step 4: Reducing

The **Reducer** adds the sales values for each product.

(Laptop, 1200)
(Mobile, 700)
(Tablet, 200)

Final Output

The result showing **total sales per product** is stored in **HDFS**.

Advantages of MapReduce

- **Efficient for Large Data** – Can process very large datasets quickly using distributed computing.
- **Parallel Processing** – Tasks are divided and processed simultaneously across multiple nodes.
- **Fault Tolerance** – Automatically handles node failures by reassigning tasks.

- **Scalability** – System performance can be improved by adding more machines.
- **Automatic Data Distribution** – Hadoop automatically distributes data and tasks across the cluster.

Limitations of MapReduce

- **Not Suitable for Real-Time Processing** – Mainly designed for batch processing.
- **High Latency** – Processing can take longer because of multiple phases.
- **Complex Programming** – Writing MapReduce programs can be difficult for beginners.
- **Inefficient for Small Files** – Hadoop works best with large datasets rather than many small files.

Conclusion

MapReduce is a distributed programming model in **Hadoop** used to process large datasets stored in **HDFS**. It works through **Map, Shuffle, and Reduce phases** to process data in parallel across a cluster. This approach provides scalability, reliability, and efficient big data processing. The **new MapReduce API** is preferred today because it offers a cleaner design and greater flexibility for modern Hadoop applications.

Practical-5

Aim: - To write a program to perform Matrix Multiplication using MapReduce technique in Java.

Introduction

Matrix multiplication is an important mathematical operation used in data analysis, machine learning, and scientific computing.

In Big Data environments, matrices can be very large. Processing them on a single machine is inefficient. Using MapReduce, matrix multiplication can be performed in parallel across multiple nodes in a Hadoop cluster.

Problem Statement

Given two matrices:

Matrix A ($m \times n$)

Matrix B ($n \times p$)

We need to compute Matrix C ($m \times p$), where:

$$C[i][j] = \sum_{k=1}^n A[i][k] \times B[k][j]$$

MapReduce Logic for Matrix Multiplication

Mapping Phase

- Each element of matrix A and B is read.
- Mapper emits intermediate key-value pairs.
- Key represents position (i, j) in result matrix.
- Value contains matrix identifier and element value.

Shuffling Phase

- Groups all values related to same (i, j).

Reducing Phase

- Multiplies corresponding values.
- Sums partial results.
- Produces final value for C[i][j].

For Example:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 1 & 3 & 2 \end{pmatrix}_{3 \times 3} \begin{pmatrix} 10 & 11 \\ 7 & 5 \\ 2 & 4 \end{pmatrix}_{3 \times 2} = \begin{pmatrix} 1*10+2*7+3*2 & 1*11+2*5+3*4 \\ 4*10+5*7+6*2 & 4*11+5*5+6*4 \\ 1*10+3*7+2*2 & 1*11+3*5+2*4 \end{pmatrix}_{3 \times 2}$$

Code:

```
import java.io.IOException;
import java.util.*;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MatrixMultiplication {

    // ----- MAPPER -----
    public static class MatrixMapper
        extends Mapper<LongWritable, Text, Text, Text> {

        private String currentMatrix = "";

        public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString().trim();
```

```

if (line.isEmpty())
    return;

if (line.startsWith("Matrix A")) {
    currentMatrix = "A";
    return;
}

if (line.startsWith("Matrix B")) {
    currentMatrix = "B";
    return;
}

String[] numbers = line.split("\\s+");

if (currentMatrix.equals("A")) {

    int row = (int) key.get(); // approximate row tracking
    for (int col = 0; col < numbers.length; col++) {
        int val = Integer.parseInt(numbers[col]);

        for (int j = 0; j < 2; j++) {
            context.write(
                new Text(row + "," + j),
                new Text("A," + col + "," + val)
            );
        }
    }
}

if (currentMatrix.equals("B")) {

    int row = (int) key.get();
    for (int col = 0; col < numbers.length; col++) {
        int val = Integer.parseInt(numbers[col]);

```

```

        for (int i = 0; i < 2; i++) {
            context.write(
                new Text(i + "," + col),
                new Text("B," + row + "," + val)
            );
        }
    }
}

// ----- REDUCER -----
public static class MatrixReducer
    extends Reducer<Text, Text, Text, Text> {

    private Map<String, Integer> resultMatrix = new TreeMap<>();

    public void reduce(Text key, Iterable<Text> values,
        Context context)
        throws IOException, InterruptedException {

        Map<Integer, Integer> A = new HashMap<>();
        Map<Integer, Integer> B = new HashMap<>();

        for (Text val : values) {
            String[] parts = val.toString().split(",");

            if (parts[0].equals("A")) {
                A.put(Integer.parseInt(parts[1]),
                    Integer.parseInt(parts[2]));
            } else {
                B.put(Integer.parseInt(parts[1]),
                    Integer.parseInt(parts[2]));
            }
        }
    }
}

```



```

    }

    int sum = 0;
    for (int k : A.keySet()) {
        if (B.containsKey(k)) {
            sum += A.get(k) * B.get(k);
        }
    }

    resultMatrix.put(key.toString(), sum);
}

@Override
protected void cleanup(Context context)
    throws IOException, InterruptedException {

    context.write(new Text("Matrix C:"), new Text(""));

    for (int i = 0; i < 2; i++) {
        String row = "";
        for (int j = 0; j < 2; j++) {
            row += resultMatrix.get(i + "," + j) + " ";
        }
        context.write(new Text(row.trim()), new Text(""));
    }
}

// ----- DRIVER -----
public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Matrix Multiplication");

    job.setJarByClass(MatrixMultiplication.class);

```

```
job.setMapperClass(MatrixMapper.class);
job.setReducerClass(MatrixReducer.class);

job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(Text.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);

FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

INPUT FILE (matrix.txt)

Matrix A:

```
1 2 3
4 5 6
```

Matrix B:

```
6 3
5 2
4 1
```

OUTPUT FILE

Matrix C:

```
28 10
73 28
```

Conclusion

Matrix multiplication using MapReduce allows large matrices to be processed in parallel across a Hadoop cluster. This approach improves scalability and performance compared to single-machine processing.

Practical-6

Aim:- To write a program to perform matrix multiplication using PySpark library in Python.

Introduction to PySpark

PySpark is the Python API for **Apache Spark**, an open-source distributed computing framework designed for large-scale data processing and analytics. It allows developers and data engineers to write Spark programs using **Python**, instead of languages like Scala or Java.

PySpark combines the power of Spark's distributed computing engine with Python's simple syntax, making it easier to process and analyze very large datasets. It is commonly used in **big data platforms, data engineering pipelines, and machine learning applications**.

Why PySpark is Used

PySpark is widely used in modern data processing systems because it provides powerful and scalable data processing capabilities.

- **Large-Scale Data Processing** – PySpark can handle massive datasets that are too large for a single machine.
- **Distributed Computing** – It processes data across multiple machines in a cluster simultaneously.
- **Efficient Big Data Analytics** – Enables fast analysis of structured and unstructured data.
- **Support for Different Data Types** – Can process structured, semi-structured, and unstructured data.
- **Machine Learning and Streaming** – Provides tools for machine learning models and real-time data streaming.

Because of these capabilities, PySpark is widely used in **data engineering, data science, and big data analytics**.

Features of PySpark

1. Distributed Processing

PySpark processes data across multiple nodes in a cluster, allowing tasks to run in parallel and significantly improving performance.

2. High Speed

Spark uses **in-memory computation**, meaning data can be stored in memory rather than repeatedly reading from disk. This makes PySpark much faster than traditional frameworks like Hadoop MapReduce.

3. Fault Tolerance

PySpark maintains a **lineage graph**, which records how data transformations were created. If a node fails, the system can automatically recompute the lost data.

4. Support for Multiple Libraries

PySpark provides several built-in libraries that extend its functionality:

- **Spark SQL** – Used for structured data processing and SQL queries.
- **MLlib (Machine Learning Library)** – Provides algorithms for machine learning tasks.
- **Spark Streaming** – Enables real-time data processing.
- **GraphX** – Used for graph processing and network analysis.

5. Easy to Use

PySpark uses Python, which has a simple syntax and is widely known among developers and data scientists. This makes it easier to write and maintain big data applications.

Matrix Multiplication using PySpark

Matrix multiplication is an important operation used in **scientific computing, machine learning, and data analysis**. It involves multiplying rows of one matrix with columns of another matrix.

If:

Matrix **A** has size (**m × n**)

Matrix **B** has size (**n × p**)

Then the resulting matrix **C** will have size (**m × p**).

Each element of matrix **C** is calculated by multiplying corresponding elements of a row from matrix **A** with a column from matrix **B**, and then summing the results.

Matrix Multiplication Process in PySpark

In PySpark, matrix multiplication is implemented using **RDDs (Resilient Distributed Datasets)** and distributed operations.

- **Conversion to RDDs** – Matrices are converted into RDDs so that data can be processed in parallel across the cluster.
- **flatMap() Transformation** – The flatMap() function is used to generate intermediate **key–value pairs** representing possible row–column combinations.
- **groupByKey() Operation** – This step groups elements with the same key and performs the **shuffle phase**, where data is transferred across nodes.
- **Multiplication and Aggregation** – Matching matrix elements are multiplied and then summed to compute each element of the resulting matrix.

Example:

```
from pyspark import SparkContext

# Initialize Spark Context
sc = SparkContext("local", "MatrixMultiplication")

# Define Matrix A (row, column, value)
A = [
    (0, 0, 2),
    (0, 1, 4),
    (1, 0, 1),
    (1, 1, 3)
]

# Define Matrix B (row, column, value)
B = [
    (0, 0, 6),
    (0, 1, 5),
    (1, 0, 2),
    (1, 1, 7)
]

n = 2 # matrix dimension

# Create RDDs
rddA = sc.parallelize(A)
rddB = sc.parallelize(B)

# Mapper Phase
mappedA = rddA.flatMap(
    lambda x: [(x[0], k), ('A', x[1], x[2])] for k in range(n)
)
mappedB = rddB.flatMap(
    lambda x: [(i, x[1]), ('B', x[0], x[2])] for i in range(n)
)

# Shuffle + Grouping Phase
grouped = mappedA.union(mappedB).groupByKey()

# Reducer Phase
def multiply(values):
    listA = []
    listB = []

    for v in values:
        if v[0] == 'A':
            listA.append(v)
        else:
            listB.append(v)

    result = 0

    for a in listA:
        for b in listB:
            if a[1] == b[1]:
                result += a[2] * b[2]

    return result

result = grouped.mapValues(multiply)
```

```
# Collect and display result
output = result.collect()

print("Result Matrix:")
for item in output:
    print("C" + str(item[0]) + " =", item[1])

sc.stop()
```

Output:

```
C(0,0) = 20
C(0,1) = 38
C(1,0) = 12
C(1,1) = 26
```

Why PySpark is Better than MapReduce

1. Faster Processing

PySpark uses **in-memory computation**, which makes it much faster than MapReduce that relies heavily on disk-based processing.

2. Simpler Programming

PySpark uses **Python with simple syntax**, making it easier to write and maintain programs compared to the complex Java-based MapReduce code.

3. Supports Real-Time Processing

PySpark can handle **streaming and real-time data processing**, while MapReduce mainly supports batch processing.

4. Less Code Required

Many operations in PySpark require **much fewer lines of code** compared to MapReduce.

5. Advanced Libraries

PySpark provides built-in libraries like **Spark SQL, MLlib, and Spark Streaming**, which are not directly available in MapReduce.

6. Better Performance for Iterative Tasks

PySpark performs better for **machine learning and iterative algorithms**, because data can stay in memory between operations.